

React Native Course Syllabus

Archer Infotech, Kothrud Pune | Classroom & live-online batches | Updated September 2026

Institute: Archer Infotech, Kothrud, Pune - IT training since 2009

Website: <https://archerinfotech.in>

Contact: +91 9850 678451 | info@archerinfotech.in

Mode: Classroom at Kothrud, Pune and live-online batches

Duration: 3 Months

Level: Intermediate

Course page: <https://archerinfotech.in/courses/mobile-app-development/react-native-training-in-pune>

Course Overview

React Native is the dominant cross-platform mobile framework for teams already deep in React / JavaScript — Pune SaaS / fintech / consumer-tech companies that ship in React (Razorpay, BharatPe, Pine Labs, Amagi, plus most Pune product startups) often pick React Native for mobile so the same engineering team can work across web and mobile. Archer Infotech's React Native training in Pune teaches the framework as it is actually used in 2026 — React Native 0.76+ with the New Architecture (Fabric renderer + TurboModules + Bridgeless mode) finally stable and default, Expo SDK 51+ as the recommended development path for most apps, modern hooks (useState / useEffect / useTransition / useOptimistic via React 19), TanStack Query for server state, plus the production tail (Expo Application Services, Codepush alternatives, EAS Build, EAS Submit). Classroom in Kothrud, online live, and weekend batches available.

Why Learn React Native in 2026

React Native is the right cross-platform choice for teams with React fluency — Indeed Pune lists 250+ active React Native Developer openings as of May 2026, with continuous hiring at Razorpay Pune, BharatPe Pune, Pine Labs Pune, Amagi, Pune-based fintech / consumer-tech / e-commerce startups, plus the Pune React Native teams of major BFSI apps. Compensation tracks Native Android within $\pm 5\%$ at every band, with the productivity multiplier of single-codebase development making senior React Native engineers attractive at startups.

What changed in 2026: the React Native New Architecture (Fabric + TurboModules + Bridgeless mode) is finally stable and default in 0.76+, eliminating the old performance-vs-Flutter gap. Expo SDK 51+ has matured into the recommended development path for most apps (replacing the old 'bare workflow' as the default). React 19 features (useTransition, useOptimistic, useFormStatus, the React Compiler) are available. Reanimated 3 + Gesture Handler 2 have stabilised the animation / gesture story. The ecosystem has consolidated — TanStack Query for server state, Zustand / Jotai for client state, React Navigation 7 for navigation, NativeWind for Tailwind-style styling.

What this means for hiring: 2026 Pune React Native JDs expect React + TypeScript fluency, the New Architecture, Expo SDK 51+, React Navigation 7, TanStack Query, plus one published app to either or both stores. Senior roles add platform-specific native modules, performance optimisation, and CI/CD with EAS.

- 250+ active React Native Developer roles on Indeed Pune (May 2026)
- New Architecture (Fabric + TurboModules + Bridgeless) finally stable in 0.76+
- Expo SDK 51+ — recommended development path for most apps
- Right framework for teams with React fluency
- Strong Pune fintech / consumer-tech / startup hiring

Who This Course Is For

This course fits you if

- Working React or Next.js developer wanting to ship mobile apps with same React fluency
- Engineering, BCS, MCA student with React background targeting cross-platform mobile
- Working JavaScript / TypeScript developer wanting to add mobile to your toolkit
- Cross-platform mobile developer wanting to add React Native to existing Flutter / native skills
- Career restarter targeting Pune startup mobile-development with web background

This course is not the right fit if

- If you have no React experience — take our React course first; this is React + a mobile platform
- If you have no JavaScript / TypeScript at the ES2020+ level
- If your goal is platform-specific deep features (iOS Apple Watch, Android Wear, ARKit / ARCore at depth)
- If you cannot put in 8–10 hours per week of practice outside class
- If you have 3+ years of production React Native experience with the New Architecture

Detailed Curriculum

Module 1: React Native Foundations & the New Architecture (Week 1)

React Native from a React-aware starting point, beginning with the differences that actually trip people up. There is no DOM — the primitives are View, Text, Image, ScrollView and FlatList — and StyleSheet is not CSS: no cascade, no inheritance beyond text, a different unit model.

The New Architecture is covered because it is the default from 0.76: the Fabric renderer, TurboModules, JSI and Bridgeless mode, and what each changed relative to the old asynchronous bridge. Understanding it is what lets you answer performance questions credibly in an interview.

- React Native primitives — View, Text, Image, ScrollView
- No DOM — how the mental model differs from web React
- StyleSheet vs CSS — no cascade, different units
- The New Architecture — Fabric, TurboModules, JSI
- Bridgeless mode and what the old bridge cost
- Platform-specific code and the Platform module
- Project structure and the Metro bundler
- Debugging with React Native DevTools

Module 2: Expo SDK, Expo Router & Development Builds (Week 2)

Expo is the recommended path for most new apps, and the course treats it as the default rather than a beginner's shortcut. The managed workflow, the Expo SDK module set, and config plugins for native configuration without ejecting.

Expo Router provides file-based navigation with typed routes, layouts and nested stacks. The critical distinction between Expo Go and a custom development build is taught early, since Expo Go silently lacks any native module outside the SDK — the cause of a great deal of beginner confusion. Prebuild and the bare workflow close the module.

- Expo managed workflow and the SDK module set
- Config plugins for native configuration
- Expo Go vs custom development builds
- Expo Router — file-based routes and layouts
- Typed routes and route groups
- expo-prebuild and the bare workflow
- Environment configuration and app variants
- Over-the-air updates with expo-updates

Module 3: Core Components, Lists & Layout (Week 3)

Building screens that behave on both platforms. Flexbox in React Native — same concepts as web with different defaults, flexDirection: column being the one that surprises everyone — plus absolute positioning, aspect ratio and percentage sizing.

Safe areas, notches, the home indicator and keyboard avoidance are handled properly rather than with magic numbers. Lists get real attention: `FlatList` and `SectionList` configuration, `keyExtractor`, `getItemLayout`, and `FlashList` for the long lists where `FlatList` starts to drop frames.

- Flexbox in React Native and its differing defaults
- Absolute positioning, `aspectRatio` and percentages
- `SafeAreaView`, insets, notches and home indicator
- Keyboard avoidance and scroll adjustment
- `FlatList` and `SectionList` configuration
- `keyExtractor`, `getItemLayout` and windowing
- Pull-to-refresh and infinite scroll
- `FlashList` for high-performance lists
- Images, caching and `resizeMode`

Module 4: Styling, Design Systems & Theming (Week 4)

Styling at the scale of a real app rather than a demo screen. The `StyleSheet` API and its performance characteristics, composition and conditional styles, and the platform-specific style patterns that keep one codebase looking correct on both.

`NativeWind` brings Tailwind syntax to React Native and is the fastest-growing choice in Pune teams; `styled-components` is covered as the alternative. The module builds a small design system — tokens, spacing scale, typography, themed components — plus dark mode, dynamic text sizing and accessibility contrast.

- `StyleSheet` API, composition and conditional styles
- Platform-specific styles and `Platform.select`
- `NativeWind` for Tailwind-style utilities
- `styled-components` as the alternative
- Design tokens, spacing scale and typography
- Building themed, reusable components
- Dark mode and `useColorScheme`
- Dynamic text sizing and accessibility contrast
- Responsive layout for tablets

Module 5: React 19 Hooks & the React Compiler on Mobile (Week 5)

Modern React applied in a mobile context. The core hooks revisited where mobile changes the calculus — `useState`, `useEffect` and its cleanup discipline, `useMemo` and `useCallback` and when they genuinely help, `useRef` for imperative handles.

React 19 additions get direct treatment: `useTransition` for keeping input responsive, `useOptimistic` for instant-feeling mutations on slow networks, and `use` for promise consumption. The React Compiler removes most manual memoisation, and the module is explicit about what it does and does not cover. Custom hooks and Context close it.

- `useState`, `useEffect` and cleanup discipline
- `useMemo` and `useCallback` — when they earn their cost
- `useRef` and imperative handles
- `useTransition` for responsive input
- `useOptimistic` for instant-feeling mutations
- The `use` hook for promise consumption
- The React Compiler and automatic memoisation
- Custom hooks for reusable logic
- Context and avoiding re-render storms

Module 6: Navigation with React Navigation 7 & Deep Linking (Week 6)

React Navigation 7 is the de-facto routing library and the layer Expo Router is built on, so it is taught directly. Native stack, bottom tabs, top tabs and drawers; nesting navigators without creating the state bugs that nesting invites; and modals and presentation styles per platform.

Params and typed navigation with TypeScript keep routes safe. Auth flows use the conditional-navigator pattern rather than imperative resets. Deep linking covers URL configuration, Android App Links, iOS Universal Links, and handling a link that arrives on a cold start.

- Native stack, bottom tabs, top tabs and drawers
- Nesting navigators without state bugs
- Modals and platform presentation styles
- Typed navigation params with TypeScript
- The conditional-navigator auth pattern
- Header configuration and screen options
- Deep link URL configuration
- App Links, Universal Links and verification
- Handling a deep link on cold start

Module 7: Forms, Validation & Server State (Week 7)

Two subjects that transfer directly from web React, which is precisely why they are worth doing well. React Hook Form with Zod for schema validation — the same stack Pune web teams use, so the skill moves in both directions — plus controlled inputs, focus management and keyboard handling on mobile.

Server state uses TanStack Query: queries and mutations, cache keys and invalidation, optimistic updates with rollback, pagination and infinite queries, and background refetch. Persisting the query cache and NetInfo awareness make the app behave sensibly offline.

- React Hook Form controllers and mobile inputs
- Zod schemas and shared web/mobile validation
- Focus management and keyboard handling
- TanStack Query — queries, mutations, cache keys
- Invalidation and refetch strategy
- Optimistic updates with rollback
- Pagination and infinite queries
- Persisting the query cache for offline reads
- NetInfo and network-aware behaviour

Module 8: Animations with Reanimated 3 (Week 8)

Animation that runs on the UI thread rather than the JavaScript one, which is the difference between smooth and janky on a mid-range Android device. Reanimated 3 shared values, useAnimatedStyle, worklets and the threading model that makes them work.

WithTiming, withSpring and withSequence cover the common motion vocabulary; layout animations and entering and exiting transitions handle list and screen changes. Scroll-driven animation, interpolation and react-native-skia for advanced graphics close the module, with reduced-motion respected throughout.

- Shared values and useAnimatedStyle
- Worklets and the UI-thread execution model
- withTiming, withSpring and withSequence
- Layout animations, entering and exiting
- Scroll-driven animation and interpolation
- Shared-element and hero-style transitions
- react-native-skia for advanced graphics
- Measuring animation performance
- Respecting reduced-motion settings

Module 9: Gestures & Native Device Capabilities (Week 9)

The features that distinguish a real app from a web page in a shell. Gesture Handler 2 — tap, pan, pinch, long press and swipeable rows — composed with Reanimated for natively responsive interaction, including a bottom sheet built rather than installed.

Device capabilities follow: camera and media library, location and background geolocation, push notifications end to end, secure storage, biometric authentication with expo-local-authentication, haptics, and the permission flows and rationale screens each of these requires on both platforms.

- Gesture Handler 2 — tap, pan, pinch, long press
- Composing gestures with Reanimated
- Swipeable rows and a hand-built bottom sheet
- Camera and media library access
- Location, geofencing and background updates
- Push notifications — permissions, tokens, handling
- AsyncStorage, MMKV and SecureStore
- Biometric authentication
- Permission flows and rationale screens

Module 10: Native Modules, TurboModules & the Expo Modules API (Week 10)

What to do when no package exists. First the judgement call — when a native module is genuinely required versus when a community package or config plugin will do — then the mechanics.

TurboModules under the New Architecture with codegen and type-safe interfaces, and the Expo Modules API as the modern authoring path: TypeScript on the React side, Swift and Kotlin on the native side, with far less boilerplate than the classic bridge. The module covers native view components, autolinking, patching a third-party package with patch-package, and testing native code.

- When a native module is genuinely required
- TurboModules, codegen and type-safe interfaces
- Expo Modules API with Swift and Kotlin
- Native view components (Fabric components)
- Autolinking and dependency resolution
- Bridging existing native SDKs
- patch-package for third-party fixes
- Testing and debugging native code
- Publishing a reusable module

Module 11: Testing, Hermes & Performance (Week 11)

Correctness and speed. Jest for unit tests, React Native Testing Library for component tests written against user-visible behaviour, mocking native modules, and end-to-end coverage with Maestro or Detox.

Performance starts with the Hermes engine — bytecode precompilation, startup cost and memory — then profiling with React Native DevTools and the Performance Monitor. The module works through the real causes of jank: re-render storms, unvirtualised lists, oversized images, expensive useEffect chains, and bundle size with Hermes source maps for production stack traces.

- Jest and React Native Testing Library
- Mocking native modules in tests
- End-to-end tests with Maestro or Detox
- The Hermes engine, bytecode and startup cost
- Profiling with React Native DevTools
- Diagnosing re-render storms
- List virtualisation and image sizing
- Bundle size and code splitting
- Source maps for production stack traces

Module 12: EAS Build, Release Engineering & Store Publishing (Week 12)

Shipping to both stores from one codebase, without owning a Mac for the iOS half. EAS Build for cloud builds with build profiles for development, preview and production; credential management, keystores and provisioning handled by EAS; and EAS Submit for automated store upload.

The store flows are covered concretely: the Play Console with its Data Safety form and target-API requirements, and App Store Connect with TestFlight and the review rejections that catch first submissions. EAS Update delivers over-the-air JavaScript changes, with an honest account of what OTA may and may not change under store policy.

- EAS Build profiles — development, preview, production
- Credential management, keystores and provisioning
- EAS Submit for automated store upload
- Play Console — Data Safety, target API, tracks
- App Store Connect, TestFlight and review causes
- App icons, splash screens and store assets
- EAS Update for over-the-air JavaScript updates
- What OTA updates may and may not change
- Versioning, build numbers and staged rollout

Module 13: Capstone Project & Placement Preparation (Weeks 12–13 + 1 week placement prep)

Full-time capstone work followed by structured interview preparation. You build and release a complete cross-platform application — Expo Router navigation, TanStack Query server state with offline persistence, Reanimated interaction, camera or location, push notifications, tests, and EAS builds submitted to both stores' test tracks.

Interview preparation matches Pune React Native panels: a React and hooks round, a React Native internals round on the New Architecture and rendering, a performance-debugging round, and a mobile system-design round. Resume, LinkedIn and GitHub polish included.

- Capstone implementation, release and README
- Submitting to internal test tracks on both stores
- React and hooks interview round
- React Native internals — Fabric, TurboModules, Hermes
- Performance-debugging round on a janky screen
- Mobile system design — offline, sync, pagination
- Resume and LinkedIn rewrite for React Native JDs
- GitHub portfolio polish
- HR mock interview and salary negotiation

Module 14: AI-Assisted Development Workflow (Final Week)

The skill every 2026 hiring panel now probes for — building real work with AI in the loop, responsibly. Learn to drive AI assistants (GitHub Copilot, Claude, Cursor, and IDE-native AI) to scaffold and accelerate the tools and stack this course covers, generate tests, explain and refactor unfamiliar code, and cut the boilerplate — while keeping you firmly in control of every decision. Heavy focus on guardrails: reviewing each AI suggestion, spotting hallucinated APIs or wrong answers, and handling licensing and data-privacy concerns. Close with a mini-project that takes a deliverable end-to-end using an AI-assisted workflow, then fold the same tooling into version control and everyday team practice.

- AI assistants — GitHub Copilot, Claude, Cursor, IDE-native AI
- Effective prompting for this course's stack — scaffolding, boilerplate, config
- AI-assisted test generation and coverage
- Explaining, refactoring, and modernising unfamiliar code with AI
- AI-driven review, error detection, and quality checks
- Generating and maintaining documentation with AI
- AI debugging — interpreting errors, logs, and failing output
- Guardrails — reviewing output, avoiding hallucinations, licensing & data privacy
- Team workflow — AI in the editor, in reviews, and in delivery pipelines
- Mini-project — a deliverable built end-to-end with an AI-assisted workflow

Capstone Projects

Production Cross-Platform App with Expo + React Navigation + TanStack Query

A complete production-grade React Native app — pick a real domain (expense tracker, food-ordering app, fitness tracker, BFSI-style mobile dashboard). Expo SDK 51+ with the New Architecture, Expo Router for navigation, TanStack Query + Zustand for state, NativeWind for styling, real APIs (public or your own backend), camera + location + push notifications, SecureStore for tokens, biometric auth. EAS Build for cloud builds, EAS Submit to both Play Store internal-test and TestFlight (Play fee ~Rs. 2,500; Apple Developer ~Rs. 8,500 / year if iOS publishing).

- Technology: Expo SDK 51+ + New Architecture
- Technology: Expo Router + React Navigation 7
- Technology: TanStack Query + Zustand
- Technology: NativeWind
- Technology: expo-camera / expo-location / expo-notifications
- Technology: EAS Build + EAS Submit

Animation-Heavy / Gesture-Rich App

A polished, animation-heavy React Native app — Reanimated 3 for UI-thread animations, Gesture Handler 2 for swipe / pinch / pan, Hero-like shared-element transitions between screens. Pick a domain that benefits from rich UX (photo viewer, music player, story-style content browser).

- Technology: Reanimated 3
- Technology: Gesture Handler 2
- Technology: Shared-element transitions
- Technology: FlatList / FlashList for lists

App with Custom Native Module via Expo Modules API

A React Native app that includes a custom native module written via the Expo Modules API — pick a feature that isn't in Expo SDK or community packages (custom Bluetooth scanning, vendor-SDK integration, advanced biometric flow). Demonstrates the patterns senior React Native interviews test for at companies that need native depth.

- Technology: Expo Modules API
- Technology: Swift (iOS native) + Kotlin (Android native)
- Technology: TypeScript bridge
- Technology: EAS Build with custom native code

Career Outcomes

React Native Developer is a high-paying mid-volume role in Pune in 2026 — Indeed Pune lists 250+ active openings, with continuous hiring at Razorpay Pune, BharatPe Pune, Pine Labs Pune, Amagi, plus the Pune-based React Native teams of major BFSI apps. Compensation tracks Native Android within $\pm 5\%$ at every band; the talent supply is thinner than Native Android, which often favours React Native engineers at offer time.

What pulls a React Native developer above the median band: React + TypeScript fluency, depth on the New Architecture (Fabric / TurboModules / Bridgeless mode), one published app to both stores via EAS, plus testing discipline (Jest + RN Testing Library + Detox).

Senior React Native Developer bands at the top end are reported as national figures (Pune-specific Indeed pages do not exist for these specific titles); Pune trends within $\pm 10\%$ of these figures.

Roles after this course

- React Native Developer
- Cross-Platform Mobile Developer
- Mobile + Web Engineer (React + RN)
- Junior Mobile Engineer
- Senior Frontend Engineer with mobile (with experience)

Indicative salary bands

- React Native Developer (Pune) - Rs. 6,80,000 per year average (source: Indeed Pune (React Native Developer))

- Junior React Native Developer (Pune entry, <2 years) - Rs. 3,80,000 – Rs. 7,00,000 per year (source: AmbitionBox Pune React Native Developer)
- Mid-level React Native Developer (Pune, 3–5 years) - Rs. 11,00,000 – Rs. 18,00,000 per year (source: Glassdoor Pune React Native Developer)
- Senior React Native Developer (Pune, 5–8 years) - Rs. 18,00,000 – Rs. 30,00,000 per year (source: Glassdoor Pune Senior React Native Developer)
- Lead React Native Engineer (national, 8+ years) - Rs. 28,00,000 – Rs. 48,00,000 per year (source: 6figr India Lead RN Engineer (Pune ±10%))

Companies hiring for these skills

- Razorpay (Pune)
- BharatPe (Pune)
- Pine Labs (Pune)
- Amagi
- Drip Capital
- Innovaccer (Pune)
- Whatfix (Pune)
- Bajaj Finserv (mobile teams)
- Pune-based fintech / consumer-tech startups
- TCS
- Infosys
- Cognizant
- Capgemini

Batch Modes and Duration

Duration: 11 weeks of structured curriculum plus 1 week of capstone and interview preparation (~3 months total)

Classroom

- Location: Archer Infotech, Kothrud, Pune
- Morning batch — 10:00 to 13:00
- Evening batch — 18:00 to 21:00

Live online

- Same hours as classroom batches
- Recordings available for review
- Zoom for live sessions
- GitHub for code reviews
- Expo Go on personal device + simulator
- Slack / WhatsApp for async Q&A

Weekend

- Saturday + Sunday, 09:00 to 13:00
- Stretches over ~5 months instead of 3.

Maximum 15 students per batch.

Fees

Course fees range Rs. 20,000 – Rs. 90,000 depending on mode and concession. EAS Build free tier covers most lab work. Play Console (~Rs. 2,500 one-time) and optional Apple Developer Program (~Rs. 8,500 / year) paid by the student.

Indicative range: Rs. 20,000 – Rs. 90,000

- Single payment with early-bird discount
- EMI in 2–3 instalments at no extra cost

- Corporate sponsorship — invoiced with GST

Placement Support

Placement support starts from week 9. By the end of the curriculum your resume highlights real published React Native apps, your GitHub has at least two production-style repositories, and you have completed at least three mock technical interviews.

We say placement support, not placement guarantee. Our support is unconditional, time-bound (six months after course completion), and includes free re-entry to a future batch's interview-prep sessions.

The placement process

- Week 9 — resume + LinkedIn rewrite for React Native JDs
- Week 10 — GitHub portfolio cleanup, store listings
- Weeks 11–12 — three rounds of mock technical interviews
- Week 12 — HR mock and salary negotiation coaching
- Post-course — referrals via our 17-year alumni network at 12+ partner companies
- Up to 6 months of continued support
- Free re-entry to future batch interview-prep sessions

Partner companies

- Razorpay (Pune)
- BharatPe (Pune)
- Pine Labs (Pune)
- Amagi
- Drip Capital
- Innovaccer (Pune)
- Whatfix (Pune)
- TCS
- Infosys
- Cognizant
- Capgemini

Prerequisites and How to Start

Prerequisites: React fluency at the level of basic hooks / components / props / state, JavaScript ES2020+ at a working level, willingness to commit 8–10 hours per week of practice. If you have done our React course or equivalent self-study, you are ready. Pure JavaScript beginners should do our React course first.

Suggested first steps

- Decide your mode — classroom, online live, or weekend
- Check the upcoming batch dates
- Book a free 30-minute counselling call
- Confirm enrolment and complete pre-course orientation (Node 22, Expo CLI install)
- Show up to day one with a laptop running 64-bit OS (macOS preferred for iOS publishing) and a smartphone for Expo Go testing

Frequently Asked Questions

How long does React Native training in Pune take at Archer Infotech?

Approximately 3 months — 11 weeks of structured curriculum plus 1 week of capstone. Weekend batch ~5 months.

What is the salary of a React Native Developer in Pune?

Indeed Pune Rs. 6.80 lakh average. Junior Rs. 3.8–7 lakh per AmbitionBox. Mid-level Rs. 11–18 lakh per Glassdoor. Senior Rs. 18–30 lakh. Lead Rs. 28–48 lakh nationally with Pune $\pm 10\%$.

Do I need React experience?

Yes — React fluency is required from day 1. If you are new to React, take our React course first.

Expo or bare React Native?

Expo SDK 51+ as the recommended path — covers ~95% of app needs without ejecting. Bare workflow covered briefly for the cases where it's needed.

Will I work on real projects?

Yes — three capstone projects with at least one published to both Play Store and TestFlight.

React Native, Flutter, or Native Android?

RN if you have React background. Flutter if cross-platform without React. Native Android for largest Pune hiring pool.

Are weekend React Native classes available in Pune?

Yes — Saturday and Sunday, 09:00–13:00, stretched over ~5 months instead of 3.

What is the fee?

Course fees range Rs. 20,000 – Rs. 90,000. Play Console (~Rs. 2,500) and optional Apple Developer (~Rs. 8,500 / year) paid by the student.

What support do I get after course completion?

Six months of placement support, referrals via our alumni network, mock interviews, salary negotiation.

Are the named trainers actually teaching?

Amol Chougule and Amol Patil personally lead every session of every batch.

Enquire or book a free demo session. Archer Infotech, Flat No. 12, Divyadarshan Housing Society, behind Kothrud Bus Stand Road, Kothrud, Pune 411038. Phone +91 9850 678451. Email info@archerinfotech.in. Web archerinfotech.in.